

# Coordination-Number Distribution Module Specification

## Integer Local-Environment Statistics with Shared Neighbor Geometry

mdstats

2026-07-11

### 1. Purpose

This document specifies integer coordination-state analysis in

`mdstats/analysis/coordination.py`

The module accepts a normalized `AtomisticFrameCollection` with trajectory or ensemble semantics, including one static frame. It counts neighbors independently in each frame and requires neither velocities nor physical time ordering.

For selected center group  $A$ , selected neighbor group  $B$ , and fixed cutoff  $r_c$ , the per-center coordination is

$$n_i^{AB}(t; r_c) = \sum_{j \in B} [\mathbf{1}_{ij}(t) < r_c],$$

with self pairs excluded.

The authoritative output is the integer matrix

$$\mathbf{N}_{ti} = n_i^{AB}(t; r_c),$$

with shape

`(n_frames, n_centers)`

All distributions and summary statistics are derived from this matrix.

### 2. Relationship to RDF and bond-angle analysis

The RDF module reports the mean cumulative coordination as a function of radius. This module reports the full probability distribution of integer coordination states at one fixed cutoff.

The recommended workflow is

```
compute_pair_rdf()
  |
  v
RDFResult.first_minimum()
  |
  v
PairCutoff.from_rdf_minimum()
  |
  v
compute_coordination_distribution()
```

The same `PairCutoff` or `PairCutoffRegistry` should be reused in bond-angle analysis so a coordination label and an angle distribution refer to exactly the same neighborhood definition.

All pair geometry is delegated to

`mdstats/analysis/_neighbors.py`

using directed center-to-neighbor lists and the strict rule

$$r_{ij} < r_c.$$

### 3. Public API

```
def compute_coordination_distribution(
    collection: AtomisticFrameCollection,
    species_a: SpeciesSelection = None,
    species_b: SpeciesSelection = None,
    *,
    cutoff: float | PairCutoff | None = None,
    cutoff_registry: PairCutoffRegistry | None = None,
    rdf_result: RDFResult | None = None,
    atom_indices_a: ArrayLike | None = None,
    atom_indices_b: ArrayLike | None = None,
    frame_start: int | None = None,
    frame_stop: int | None = None,
    frame_step: int = 1,
    minimum_options: Mapping[str, Any] | None = None,
    block_size: int = 256,
    neighbor_search_options: NeighborSearchOptions | None = None,
) -> CoordinationResult:
    ...
```

The public API supports three cutoff sources for convenience, but exactly one must be active:

1. cutoff as a raw float or PairCutoff;
2. cutoff\_registry, from which the  $A - B$  pair is required;
3. rdf\_result, converted internally to PairCutoff using the first minimum.

The normalized internal representation is always one PairCutoff.

### 4. Parameters

#### 4.1 collection

Required fields:

- fixed atomic identities and canonical indexing;
- cells and PBC flags;
- positions for every frame.

Trajectory continuity is not required.

#### 4.2 Center and neighbor selections

species\_a selects centers and species\_b selects candidate neighbors. In the implemented two-group coordination API, each resolved group must contain one unique chemical species because the result stores one species-pair PairCutoff. Typical selections are:

```
species_a="Na"
species_b="O"
```

Optional explicit index arrays may restrict either selection. If species and indices are both supplied, every index must belong to the requested species. A multi-species selector such as ("Si", "Al") is rejected here; use a bond-angle CoordinationCondition when different species-specific cutoffs must be combined.

Selections must be identical or disjoint. Partial overlap is rejected.

#### 4.3 Cutoff sources

##### Manual float

```
cutoff=3.2
```

is converted to a manual PairCutoff for the selected species pair.

### PairCutoff

cutoff=na\_o\_cutoff

preserves cutoff provenance.

### Registry

cutoff\_registry=cutoffs

requires the canonical selected pair from the registry.

### RDF result

rdf\_result=rdf\_na\_o

calls

```
PairCutoff.from_rdf_minimum(  
    rdf_result,  
    minimum_options=minimum_options,  
)
```

The RDF pair selection must match the coordination selection, independent of pair ordering. Frame-subset differences are allowed but produce a provenance warning.

## 4.4 Frame selection

frame\_start, frame\_stop, and frame\_step use Python slicing semantics. At least one frame is required and frame\_step > 0.

## 4.5 block\_size

Center-block size used by the dense oracle. It changes temporary memory but not the mathematical estimator.

## 4.6 neighbor\_search\_options

Optional NeighborSearchOptions. None uses the semantics-aware production default: eligible trajectories may use deformation-aware Verlet reuse, while single-frame selections and independent ensembles remain stateless. The per-atom/per-frame integer matrix must be exactly identical under dense, stateless cell-list, and cached execution.

## 5. Result object

```
@dataclass(slots=True)  
class CoordinationResult:  
    species_a: str  
    species_b: str  
  
    pair_cutoff: PairCutoff  
  
    coordination_values: NDArray[np.int32]  
    counts: NDArray[np.int64]  
    probabilities: NDArray[np.float64]  
  
    per_atom_per_frame: NDArray[np.int32]  
    per_frame_mean: NDArray[np.float64]  
    per_frame_std: NDArray[np.float64]  
    per_atom_mean: NDArray[np.float64]  
    per_atom_std: NDArray[np.float64]  
  
    atom_indices_a: NDArray[np.int64]  
    atom_indices_b: NDArray[np.int64]  
    frame_indices: NDArray[np.int64]  
  
    mean: float  
    std: float  
    variance: float
```

```
metadata: dict[str, Any] = field(default_factory=dict)
```

The authoritative cutoff object is `pair_cutoff`. The implementation also provides read-only convenience views:

```
result.cutoff
result.cutoff_source
result.cutoff_feature
```

`cutoff_feature` reconstructs the RDF minimum feature when provenance is available and otherwise returns `None`.

## 5.1 Authoritative raw matrix

```
per_atom_per_frame[t, i]
```

is the number of selected neighbors around center `atom_indices_a[i]` in frame `frame_indices[t]`.

Every element is a nonnegative integer.

## 5.2 Distribution

Let  $N_f$  be the number of selected frames and  $N_A$  the number of centers. The count of coordination state  $q$  is

$$H(q) = \sum_{t=1}^{N_f} \sum_{i=1}^{N_A} \mathbf{1}[N_{ti} = q].$$

The probability is

$$P(q) = \frac{H(q)}{N_f N_A},$$

with

$$\sum_q P(q) = 1.$$

`coordination_values` is always

```
[0, 1, ..., q_max]
```

so absent intermediate states have zero count rather than disappearing from the support.

## 5.3 Summary statistics

Population statistics use `ddof=0`.

Global mean:

$$\bar{n} = \frac{1}{N_f N_A} \sum_{t,i} N_{ti}.$$

Variance:

$$\text{Var}(n) = \frac{1}{N_f N_A} \sum_{t,i} (N_{ti} - \bar{n})^2.$$

Per-frame and per-atom means and standard deviations are direct reductions of `per_atom_per_frame`.

## 5.4 Implemented convenience interface

```
result.n_frames
result.n_atoms_a
result.n_atoms_b
result.n_observations
result.probability_at(coordination)
result.most_probable_coordination
result.to_dataframe() # requires pandas
```

```
result.save_npz(path)
restored = CoordinationResult.load_npz(path)
```

The NPZ round trip includes the serialized PairCutoff, raw integer matrix, summary arrays, selections, frame indices, and JSON-safe metadata.

## 6. High-level algorithm

```
resolve frames and selections
|
resolve exactly one PairCutoff
|
validate cutoff over selected frames
|
for each frame:
    build directed shared neighbor list
    coordination = difference of CSR offsets
|
assemble per_atom_per_frame
|
derive histogram and summary statistics
|
construct CoordinationResult
```

Pseudo-code:

```
for row, frame in enumerate(frame_indices):
    neighbors = build_neighbor_list(
        collection,
        frame_index=frame,
        center_indices=A,
        candidate_neighbor_indices=B,
        cutoff=pair_cutoff,
        pair_counting=DIRECTED,
    )

    per_atom_per_frame[row] = diff(neighbors.offsets)
```

The coordination module does not call `find_mic` directly and does not maintain its own blocked pair-distance implementation.

## 7. Combined-species coordination

The implemented ordinary coordination function requires one unique species in each group. It intentionally rejects a multi-species neighbor selector such as

```
species_b=("Si", "Al")
```

because one PairCutoff cannot represent distinct Na-Si and Na-Al, or O-Si and O-Al, shell boundaries.

Combined-species coordination is implemented for bond-angle filtering through `CoordinationCondition`. That path obtains one neighbor list per species from a `PairCutoffRegistry` and counts the set union of canonical atom indices. For example,

```
CoordinationCondition.exact(("Si", "Al"), 2)
```

can use distinct O-Si and O-Al cutoffs. A future generalized coordination result may expose the same multi-cutoff union as a standalone distribution, but the current two-group API does not.

## 8. Cutoff validation and provenance

Every cutoff must satisfy

$$0 < r_c \leq r_{\text{safe}},$$

where  $r_{\text{safe}}$  is calculated over all selected frames by the shared neighbor layer.

The result retains the full `PairCutoff`, including source metadata. This makes manual and RDF-derived analyses reproducible and allows downstream angle analysis to verify that it uses the same pair definition.

A low-confidence RDF minimum does not invalidate the numerical calculation, but the coordination function should emit a warning.

## 9. Relationship to the RDF cumulative curve

At the same center/neighbor selections, frame subset, and cutoff,

$$\bar{n}$$

from `CoordinationResult` should agree with the RDF cumulative coordination value evaluated at  $r_c$ , up to radial-bin discretization.

The integer coordination matrix contains more information. Two systems can have the same mean coordination but different distributions:

System 1: all centers have coordination 4

System 2: half have 3 and half have 5

Both have mean four, but only the second contains coordination heterogeneity.

## 10. Frame semantics

For a trajectory, `per_atom_per_frame` records temporal coordination histories, although this module does not compute autocorrelations or lifetimes.

For an ensemble, rows are independent samples and no temporal interpretation is allowed.

For one frame, probabilities describe the center-atom population in that single structure.

## 11. Edge cases and warnings

The function rejects:

- absent or empty center/neighbor groups;
- invalid explicit indices;
- partially overlapping selections;
- multiple simultaneous cutoff sources;
- incompatible RDF or registry pairs;
- unsafe, nonfinite, or nonpositive cutoffs;
- invalid frame selection or block size;
- malformed cells or coordinates.

The implementation emits `CoordinationFrameMismatchWarning` when an RDF-derived cutoff was estimated from a different frame selection. Other sampling concerns are not emitted as heuristic warnings. RDF feature confidence remains available through `result.pair_cutoff.source_metadata` and `result.cutoff_feature` for explicit user inspection.

A coordination count is defined by the supplied cutoff, not by an intrinsic chemical-bond oracle. Broad shells, fluctuating liquids, and overlapping species pairs require physical judgment.

## 12. Shared-neighbor refactor requirements

The implemented module delegates minimum-image geometry, safe-cutoff validation, blocked pair enumeration, self exclusion, and CSR construction to `_neighbors.py`.

The coordination module retains observable-specific selection classification, cutoff-source resolution, unique-species validation, RDF compatibility checks, statistical reductions, serialization, and plotting-facing result methods.

## 13. Testing requirements

Tests must cover:

1. Hand-counted orthogonal structures.
2. Periodic and triclinic neighbor crossings.
3. Identical selections with self exclusion.
4. Rejection of mixed-species groups in the two-group API.
5. Manual, registry, and RDF-derived cutoff sources.
6. Strict exclusion at `distance == cutoff`.
7. Agreement with shared CSR offset differences.
8. Agreement of the global mean with RDF cumulative coordination.
9. Variable-cell collections.
10. Trajectory, ensemble, and single-frame inputs.
11. Cutoff-provenance serialization.
12. RDF/coordination frame-mismatch warnings.

## 14. Deliberate non-goals

The module does not yet implement:

- coordination autocorrelation or residence lifetimes;
- hysteretic bond definitions;
- per-species multi-cutoff unions in the ordinary public function;
- Voronoi or fixed-nearest-neighbor coordination;
- coordination-conditioned angle calculations inside this module;
- ring or site-occupancy classification.

Those features should build on the shared neighbor layer and, where appropriate, `CoordinationCondition`.

## 15. Example workflow

```
rdf = compute_pair_rdf(
    collection,
    species_a="Na",
    species_b="O",
    r_max=6.0,
)

na_o_cutoff = PairCutoff.from_rdf_minimum(rdf)

coordination = compute_coordination_distribution(
    collection,
    species_a="Na",
    species_b="O",
    cutoff=na_o_cutoff,
)
```

The result may then be used to select a physically meaningful filter for O-Na-O angle statistics:

```
filter_sixfold = CoordinationCondition.exact("O", 6)
```

## 16. Reproduction checklist

An independent implementation reproduces this module when it:

1. uses directed shared neighbor lists;
2. counts coordination from CSR row lengths;
3. applies one fixed pair cutoff across selected frames;
4. excludes self pairs and applies `distance < cutoff`;
5. retains the full integer atom-frame matrix;
6. derives every statistic from that matrix;
7. preserves cutoff provenance in `PairCutoff`;
8. behaves identically for trajectory and ensemble semantics when the frames are otherwise identical.

## 17. Stage S4 execution policy

One analysis-local executor persists across selected frames. Backend and cache diagnostics are stored in `metadata["neighbor_search"]`; they do not alter the coordination matrix, distribution, or moments. The normative execution contract is `docs/specs/analysis/neighbor_search_spec.md`.